

UNIVERSIDAD DE EL SALVADOR

Facultad Multidisciplinaria De Occidente
Departamento de Ingeniería y Arquitectura

Ingeniería En Desarrollo De Software/Educación En Línea
LOGICA DE PROGRAMACION
SEMESTRE 1 / 2025



ACTIVIDAD: GUÍA DE LABORATORIO NÚMERO 6

Clasificación de las listas, Operaciones en Listas e Introducción a POO.

Nombre: _____, Carnet: _____
Fecha: _____, Grupo teórico: _____

Objetivo: Conocer los conceptos y el uso de las listas, clasificación de las listas, operaciones en listas en introducción a POO, además de exponer una breve introducción al paradigma orientado a Objetos en C++.

Indicación general: Leer detenidamente cada una de las tres partes en que está dividida la actividad y brindar respuestas en las que se solicite.

Parte 1 - Introducción al Paradigma Orientado a Objetos

Indicación: leer detenidamente el contenido de introducción al paradigma orientado a objetos y los elementos que este contiene.

Un paradigma de programación, esto es, un modelo o un estilo de programación que proporciona unas guías acerca de cómo trabajar con él.

Con el paradigma de POO lo que se busca es dejar de centrarse en la lógica pura de los programas, para comenzar a pensar en objetos, lo que forma la base de dicho paradigma. Esto ayuda bastante en sistemas grandes, pues en lugar de pensar en funciones, se piensa en las relaciones o interacciones de los distintos elementos del sistema.

En POO se hacen clases y luego se crean objetos a partir de dichas clases, que constituyen el modelo a partir del que se estructuran los datos y los comportamientos. Y es que quizá el concepto más importante sea distinguir entre clase y objeto.

Una **clase** es una plantilla, que define de modo genérico cómo serán los objetos de un determinado tipo. Por ejemplo, una clase para representar a animales puede denominarse 'animal' y disponer de una serie de atributos, como 'nombre' o 'edad' (que habitualmente son propiedades), y una serie con los comportamientos que estos pueden tener, como caminar o comer, y que a su vez se implementan como métodos de la clase (funciones).

Un ejemplo sencillo de un **objeto**, podría ser un animal. Un animal tiene una edad, por lo que se crea un nuevo atributo de 'edad', y, además, puede envejecer, por lo que se define un nuevo método.

Con la clase pueden crearse instancias de un objeto, cada uno de ellos con sus atributos definidos independientemente. Con esto se puede crear un gato llamado Pepe, con 5 años de edad, y otro animal, tipo perro y llamado Lucho, con una edad de 3 años. Los dos están definidos por la clase animal, pero son dos instancias diferentes. Así, llamar a sus métodos puede tener resultados distintos. Los dos comparten la lógica, pero cada uno tiene su estado de manera independiente.

Todo esto son herramientas que ayudan a escribir un código mejor, más limpio y reutilizable.

Ventajas de Programación Orientada a Objetos

- Reutilización del código.
- Convierte cosas complejas en estructuras simples reproducibles.
- Evita la duplicación de código.
- Permite ~~trabaja~~ trabajar en equipo gracias al encapsulamiento, puesto que minimiza la posibilidad de duplicar funciones cuando distintas personas trabajan sobre un mismo objeto al mismo tiempo.
- Protege la información mediante la encapsulación, pues solo se puede acceder a los datos del objeto mediante propiedades y métodos privados.
- La abstracción nos permite construir sistemas más complejos y de un modo más sencillo y organizado

Parte 2 – Introducción a iostream: cout y endl

Indicación: leer detenidamente el contenido de introducción a iostream **cout** y **endl** y las características que tienen.

La biblioteca de entrada/salida

La biblioteca de entrada/salida (biblioteca io) es parte de la biblioteca estándar de C++ que se ocupa de la entrada y salida básicas. Usaremos la funcionalidad en esta biblioteca para obtener entradas desde el teclado y enviar datos a la consola. La parte **io** de **iostream** significa entrada/salida.

Para usar la funcionalidad definida dentro de la biblioteca **iostream**, debemos incluir el encabezado **iostream** en la parte superior de cualquier archivo de código que use el contenido definido en **iostream**, así:

```
1 #include <iostream>
2
3 // rest of code that uses iostream functionality here
```

std::cout

La biblioteca **iostream** contiene algunas variables predefinidas para que las usemos. Uno de los más útiles es **std::cout**, que nos permite enviar datos a la consola para que se impriman como texto. **cout** significa "salida de caracteres".

```
1 #include <iostream> // for std::cout
2
3 int main()
4 {
5     std::cout << "Hello world!"; // print Hello world! to console
6
7     return 0;
8 }
```

En este programa, hemos incluido **iostream** para que tengamos acceso a **std::cout**. Dentro de nuestra función principal, usamos **std::cout**, junto con el operador de inserción (<<), para enviar el texto **Hello world!** a la consola para ser impreso.

std::endl

¿Qué esperarías que imprimiera el siguiente programa?

```
1 #include <iostream> // for std::cout
2
3 int main()
4 {
5     std::cout << "Hi!";
6     std::cout << "My name is Alex.";
7     return 0;
8 }
```

Te sorprenderías al saber que el resultado es:

```
Hi!My name is Alex.
```

Las declaraciones de salida separadas no dan como resultado líneas de salida separadas en la consola.

Si queremos imprimir líneas separadas de salida en la consola, debemos decirle a la consola cuándo mover el cursor a la siguiente línea.

Una forma de hacerlo es usar `std::endl`. Cuando se genera con `std::cout`, `std::endl` imprime un carácter de nueva línea en la consola (haciendo que el cursor vaya al comienzo de la siguiente línea). En este contexto, `endl` significa "línea final".

```
1 #include <iostream> // for std::cout and std::endl
2
3 int main()
4 {
5     std::cout << "Hi!" << std::endl; // std::endl will cause the cursor to move to the next
6     std::cout << "My name is Alex." << std::endl;
7
8     return 0;
9 }
```

El código anterior imprime:

```
Hi!
My name is Alex.
```

Parte 3- Desarrollar los siguientes puntos, ponderación de actividad 100%

Indicación: Brindar solución a los siguientes puntos y desarrolla software si es parte de lo solicitado, tomando como referencia los tópicos abordados en clase.

1. Las listas se pueden dividir en cuatro categorías: simplemente enlazadas, doblemente enlazadas, circular simplemente enlazada y circular doblemente enlazada. Mencione un ejemplo de uso de cada uno de los tipos de listas antes mencionadas.
2. Mencione las operaciones que se pueden realizar con los tipos de listas, además de exponer un ejemplo, programa en C++, por cada uno de ellos.
3. Con sus palabras defina que es la programación orientada a objetos y sus ventajas versus la programación estructurada (se pide realizarlo en más de 5 líneas).